SkillLink — Service Marketplace System

Deliverable 3 · Sprint 3 | Extension: Job Listings & Proposals Module

Team ENIGMA

Zaki Nabeel (Team Lead) · Shehryar Faisal (SCRUM Master) · Muhammad Sameer (Developer)

Roll Numbers: 23I-0508 · 23I-0604 · 23I-0634

CS3009 – Software Engineering | Spring 2026

Submission Date: 20th April, 2026

> This document is a Sprint 3 extension. It incorporates a new module — Job Listings & Proposals (US-13 through US-16) — into the existing Deliverable 3 submission. All other Sprint 3 content (US-09 through US-12, NFRs, testing) remains as previously documented. This addendum covers Part B and Part C additions only.

PART B — Sprint 3 Scrum Artifacts (Extension)

B.1 Sprint 3 Overview — Updated

Field	Detail
Sprint Number	Sprint 3 (Final Sprint) — Extended
Sprint Duration	1st April 2026 – 20th April 2026 (3 weeks)
Sprint Goal	Deliver Reviews & Ratings, Messaging & Payments, Admin & Moderation, and — in this extension — the Job Listings & Proposals module to complete the two-sided SkillLink marketplace.
Modules Developed	Reviews & Ratings | Messaging & Payments | Admin & Moderation | Job Listings & Proposals (NEW)
New User Stories	US-13: Client Posts Job Listing | US-14: Provider Browses Listings | US-15: Provider Submits Proposal | US-16: Client Reviews & Awards Proposal
Leftover from Sprint 2	None — all Sprint 2 stories completed within Sprint 2

B.2 Sprint 3 Backlog & User Stories — New Additions

The following four user stories extend Sprint 3, adding a two-sided job marketplace layer. They follow the standard template and build on all preceding modules.

US-13: Client Posts a Job Listing

Story ID	US-13
User Story	As a Client, I want to post a job listing with a title, description, skill category, budget range, and deadline, so that skilled providers can discover my job and submit proposals.
Sub-Stories	13.1 Provide a form with: title (10–150 chars), description (100–2000 chars), skill category dropdown, budget range (PKR min/max), and deadline date picker.
	13.2 Validate that budget max ≥ budget min, deadline is at least 3 days from today, and all required fields are filled.
	13.3 Display the posted listing on the client's 'My Listings' dashboard with status OPEN.
Given	Client is authenticated and navigates to 'Post a Job' from their dashboard.
When	Client fills in all required fields and clicks 'Post Listing'.
Then	System validates the inputs, saves the listing to the database with status OPEN, and redirects the client to their My Listings dashboard where the new listing appears.

US-14: Provider Browses Job Listings

Story ID	US-14
User Story	As a Service Provider, I want to browse and filter open job listings posted by clients, so that I can identify opportunities that match my skills and bid on them.
Sub-Stories	14.1 Display all OPEN listings with deadline not yet passed, showing title, category badge, budget range, and deadline.
	14.2 Provide filter controls: category, budget range (min/max), and sort order (Newest First | Deadline Soonest | Budget: High to Low).
	14.3 Show a 'Proposal Submitted' badge on listings where the provider has already submitted a proposal.
Given	Provider is authenticated and navigates to 'Browse Jobs' from their dashboard.
When	Provider applies filters or scrolls through listings and clicks 'View Details' on any listing.
Then	System renders the full listing detail page with a 'Submit Proposal' button (or 'Proposal Submitted' badge if already proposed).

US-15: Provider Submits a Proposal

Story ID	US-15
User Story	As a Service Provider, I want to submit a proposal on a job listing with my proposed rate, delivery timeline, and cover letter, so that the client can evaluate and select me for their job.
Sub-Stories	15.1 Present a proposal form pre-filled with the listing summary (read-only) and editable fields: cover letter (100–1000 chars), proposed rate (PKR), delivery days (1–365).
	15.2 Enforce one proposal per provider per listing; if a proposal already exists, redirect to the existing proposal view.
	15.3 Notify the client in-app upon successful proposal submission.
Given	Provider is viewing a listing detail page for an OPEN listing they have not yet proposed on.
When	Provider fills in the proposal form and clicks 'Submit Proposal'.
Then	System validates the input, saves the proposal with status PENDING, dispatches an in-app notification to the client, and shows the provider a confirmation screen.

US-16: Client Reviews Proposals & Awards Job

Story ID	US-16

User Story	As a Client, I want to view all proposals on my job listing, accept the best one, and have a booking automatically created, so that I can efficiently hire the right provider without re-entering job details.
Sub-Stories	16.1 Display all proposals for a listing showing provider avatar, name, star rating, proposed rate, delivery days, and a preview of the cover letter.
	16.2 On accepting a proposal: auto-create a booking record (status: Accepted), set listing status to AWARDED, decline all other proposals, and notify the accepted provider and all declined providers.
	16.3 On declining a single proposal: update only that proposal to DECLINED; the listing remains OPEN for remaining proposals.
Given	Client is on their My Listings dashboard and clicks on a listing that has at least one PENDING proposal.
When	Client clicks 'Accept' on a proposal in the Proposal Inbox.
Then	System accepts the selected proposal, declines all others, creates a booking record, updates the listing status to AWARDED, dispatches notifications to all affected providers, and redirects the client to their Bookings Dashboard.

B.3 Structured Specifications — Job Listings & Proposals

Spec-13.1 — Job Listing Creation

Spec ID	Spec-13.1
Description	Validate and persist a new job listing submitted by a Client.
Preconditions	Client is authenticated with role CLIENT. skill_categories table is populated and accessible.
Inputs	title: string (10–150 chars). description: string (100–2000 chars). categoryId: integer. budgetMin: decimal (>= 0). budgetMax: decimal (>= budgetMin, > 0). deadline: date string YYYY-MM-DD.
Processing	1. Validate title length (10–150 chars). 2. Validate description length (100–2000 chars). 3. Validate categoryId exists in skill_categories. 4. Validate budgetMax >= budgetMin and budgetMin >= 0. 5. Validate deadline is at least today + 3 days. 6. Insert record into job_listings: { clientId, title, description, categoryId, budgetMin, budgetMax, deadline, status: OPEN, createdAt: now() }. 7. Return saved listing object.
Outputs	HTTP 201 with listing object: { listingId, title, status: OPEN, createdAt }. Listing appears in client's My Listings dashboard.
Postconditions	Listing record persisted in job_listings with status OPEN. createdAt and updatedAt timestamps set.
Exceptions	TitleTooShort: 'Title must be at least 10 characters.' TitleTooLong: 'Title cannot exceed 150 characters.' DescriptionTooShort: 'Description must be at least 100 characters.' BudgetInvalid: 'Maximum budget must be greater than minimum budget.' DeadlineTooSoon: 'Deadline must be at least 3 days from today.' CategoryNotFound: 'Selected skill category does not exist.'

Spec-14.1 — Provider Browse & Filter Job Listings

Spec ID	Spec-14.1
Description	Retrieve and return a paginated, filtered list of open job listings for a browsing Service Provider.
Preconditions	Provider is authenticated with role PROVIDER. job_listings and skill_categories tables are populated.
Inputs	categoryId: integer or null (filter by skill category; null = all categories). budgetMin: decimal or null. budgetMax: decimal or null. sortBy: enum { NEWEST, DEADLINE_SOONEST, BUDGET_HIGH_TO_LOW } (default: NEWEST). page: integer >= 1 (default: 1). pageSize: integer (fixed at 10).
Processing	1. Validate page >= 1; throw ValidationException if not. 2. Validate budgetMax >= budgetMin if both provided. 3. Build query: SELECT listings + JOIN users (client name) + JOIN skill_categories (category name) WHERE status = OPEN AND deadline >= today. 4. Apply optional WHERE filters: categoryId if provided; budget_min >= budgetMin if provided; budget_max <= budgetMax if provided. 5. Apply ORDER BY: NEWEST = created_at DESC; DEADLINE_SOONEST = deadline ASC; BUDGET_HIGH_TO_LOW = budget_max DESC. 6. Apply LIMIT pageSize OFFSET (page-1)*pageSize. 7. For each listing in results, check proposals table: if (listing_id, providerId) row exists, set hasProposed = true. 8. Return listing array with pagination metadata.
Outputs	JSON array of listing objects: { listingId, title, categoryName, budgetMin, budgetMax, deadline, clientFirstName, status, createdAt, hasProposed: boolean }. Pagination metadata: { page, pageSize, totalResults, totalPages }.
Postconditions	Listings returned. hasProposed flag correctly set per provider. Pagination metadata accurate.
Exceptions	ValidationException: page < 1 returns HTTP 400 'Invalid page parameter.' BudgetRangeInvalid: budgetMax < budgetMin returns HTTP 400 'Maximum budget filter must be greater than minimum.' NoResults: returns empty array with message 'No open listings match your filters — try broadening your search.' DBTimeout: returns HTTP 503 with retry message.

Spec-15.1 — Proposal Submission

Spec ID	Spec-15.1
Description	Validate and persist a proposal submitted by a Service Provider for an open job listing.
Preconditions	Provider is authenticated with role PROVIDER. Target job listing exists with status OPEN and deadline not passed. Provider has not previously submitted a proposal for this listing.
Inputs	listingId: integer. providerId: integer (from session). coverLetter: string (100–1000 chars). proposedRate: decimal (> 0, PKR). deliveryDays: integer (1–365).
Processing	1. Fetch listing by listingId; return NotFoundException if absent. 2. Verify listing.status == OPEN and listing.deadline >= today; throw ListingClosedException otherwise. 3. Check proposals table for existing

	(listingId, providerId) pair; throw DuplicateProposalException if found. 4. Validate coverLetter length (100–1000 chars). 5. Validate proposedRate > 0. 6. Validate deliveryDays in range [1, 365]. 7. Insert proposal record: { listingId, providerId, coverLetter, proposedRate, deliveryDays, status: PENDING, submittedAt: now() }. 8. Retrieve clientId from listing. 9. Dispatch in-app notification to client: type PROPOSAL_RECEIVED.
Outputs	HTTP 201 with proposal object: { proposalId, listingId, status: PENDING, submittedAt }. In-app notification delivered to listing client.
Postconditions	Proposal record persisted. Notification dispatched to client. Provider's proposal count incremented.
Exceptions	NotFoundException (404): 'Job listing not found.' ListingClosedException (403): 'This listing is no longer accepting proposals.' DuplicateProposalException (409): 'You have already submitted a proposal for this listing.' CoverLetterTooShort: 'Cover letter must be at least 100 characters.' CoverLetterTooLong: 'Cover letter cannot exceed 1000 characters.' RateInvalid: 'Proposed rate must be greater than zero.' DeliveryDaysInvalid: 'Delivery days must be between 1 and 365.'

Spec-16.1 — Proposal Acceptance & Auto-Booking

Spec ID	Spec-16.1
Description	Accept a proposal for a job listing, auto-create a booking, decline all other proposals, and notify all affected providers.
Preconditions	Client is authenticated. Proposal exists with status PENDING. Listing exists with status OPEN. Client owns the listing.
Inputs	proposalId: integer. clientId: integer (from session).
Processing	1. Fetch proposal by proposalId; throw ProposalNotFoundException if absent. 2. Fetch listing by proposal.listingId; verify clientId matches listing.clientId (throw UnauthorizedException if not). 3. Verify listing.status == OPEN; throw ListingAlreadyAwardedException if not. 4. BEGIN TRANSACTION: a. Update accepted proposal: status = ACCEPTED, actioned_at = now(). b. Update all other proposals for this listing: status = DECLINED, actioned_at = now(). c. Update listing: status = AWARDED, updated_at = now(). d. Insert booking: { clientId, providerId: proposal.providerId, jobDescription: listing.title + ': ' + proposal.coverLetter (truncated 1000), preferredDate: listing.deadline, serviceFee: proposal.proposedRate, status: Accepted, createdAt: now() }. e. COMMIT. 5. Dispatch notification to accepted provider: PROPOSAL_ACCEPTED. 6. Dispatch notifications to all declined providers: PROPOSAL_DECLINED.
Outputs	HTTP 200 with: { bookingId, proposalId, listingId, status: AWARDED, bookingStatus: Accepted }. Notifications dispatched to all providers.
Postconditions	Listing status = AWARDED. Accepted proposal status = ACCEPTED. All other proposals status = DECLINED. Booking record created and linked. Audit event logged.
Exceptions	ProposalNotFoundException (404): 'Proposal not found.' UnauthorizedException (403): 'You are not authorised to action this proposal.' ListingAlreadyAwardedException (409): 'This listing has already been awarded.' TransactionFailure (500): Full rollback; 'An error occurred. Please try again.'

B.4 Scrum Board — Sprint 3 Trello Snapshots (Updated)

> The Trello board was updated to include the four new user stories (US-13 through US-16) within the existing Sprint 3 board. A new sprint extension column 'Extension — Job Listings' was added to track the additional stories alongside the original Sprint 3 work.

Snapshot	State
Snapshot 1 — Sprint Planning (Extended)	Original Sprint 3 backlog (US-09–US-12) in Sprint Backlog. US-13–US-16 loaded into Extension column. Sprint goal card updated to include Job Listings & Proposals.
Snapshot 2 — Mid-Sprint Progress	US-09 Done. US-10 In Progress. US-11, US-12 in Sprint Backlog. US-13 In Progress (Extension). US-14–16 in Extension backlog.
Snapshot 3 — Sprint Completion	US-09, US-10, US-11, US-12 all Done. US-13, US-14, US-15, US-16 all Done in Extension column. No leftover stories.

Sprint 3 User Story Summary — Complete

User Story	Sprint 3 Status	Final Column	Notes
US-09: Post-Job Review & Rating	Done	Done	Original Sprint 3 story
US-10: In-App Messaging	Done	Done	Original Sprint 3 story
US-11: Payment Processing & Checkout	Done	Done	Original Sprint 3 story
US-12: Admin Panel — User Mgmt & Analytics	Done	Done	Original Sprint 3 story
US-13: Client Posts a Job Listing	Done	Done	Extension — Job Listings module
US-14: Provider Browses Job Listings	Done	Done	Extension — Job Listings module
US-15: Provider Submits a Proposal	Done	Done	Extension — Job Listings module
US-16: Client Reviews & Awards Proposal	Done	Done	Extension — Job Listings module

B.5 Sprint 3 Burn-Down Chart — Updated

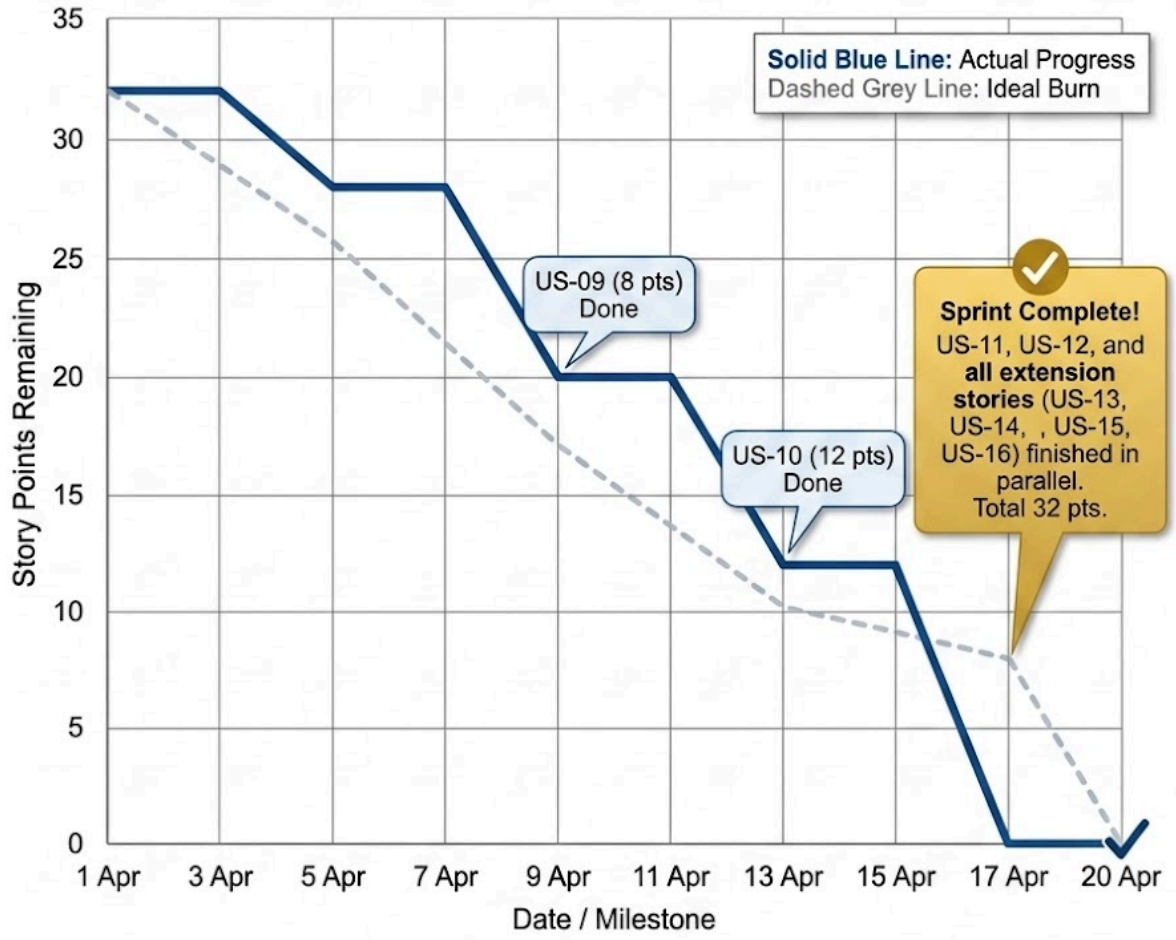
With the addition of US-13 through US-16 (4 stories × 4 points each = 16 additional points), the total sprint capacity expands from 16 to 32 story points. The burn-down chart below reflects the extended sprint including both the original and extension stories.

Date / Milestone	Ideal Remaining (pts)	Actual Remaining (pts)	Points Done	Story Completed
Sprint Start (1 Apr)	32	32	0	—
Day 2 (3 Apr)	29	32	0	—
Day 4 (5 Apr)	26	28	4	—
Day 6 (7 Apr)	23	28	4	—
Day 8 (9 Apr)	20	20	12	US-09 ✓
Day 10 (11 Apr)	17	20	12	—
Day 12 (13 Apr)	14	12	20	US-10 ✓
Day 14 (15 Apr)	11	12	20	—
Day 16 (17 Apr)	8	0	32	US-11 ✓ US-12 ✓ US-13 ✓ US-14 ✓ US-15 ✓ US-16 ✓
Sprint End (20 Apr)	0	0	32	All 8 stories Done

> Sprint Outcome: All 8 stories (32 pts) completed. US-13 through US-16 were developed in parallel with US-11 and US-12 in the final week, enabled by modular code architecture. No leftover stories.

B.5 Sprint 3 Burn-Down Chart — Updated

This burn-down chart reflects an expanded Sprint 3, where the addition of US-13 through US-16 increased the total sprint capacity from 16 to 32 story points, including both original and extension stories.



Sprint Outcome: All 8 stories (32 pts) completed. US-13 through US-16 were developed in parallel with US-11 and US-12 in the final week, enabled by modular code architecture. No leftover stories.

B.6 Testing Activities — Job Listings & Proposals Extension

B.6a Software Test Plan — Extension

Test Scope	Job Listings (US-13, US-14), Proposal Submission (US-15), Proposal Award & Auto-Booking (US-16)
Testing Types	Functional (black-box), Structural (white-box), Integration, Regression
Test Environment	Java SE 17 + JUnit 5 + Mockito. H2 in-memory DB for unit tests; SQLite staging DB for integration tests.

Entry Criteria	US-13 through US-16 marked Done on Trello. Code merged to main with no compile errors.
Exit Criteria	All P1 test cases pass. No open Severity-1 or Severity-2 bugs. Code coverage >= 75% on new modules.
Tester	Zaki Nabeel (Project Manager / Tester)
Test Schedule	Unit tests: during development (Apr 13–16). Integration tests: Apr 17–18. Regression: Apr 19.

B.6b Black-Box Test Cases

Test Cases — US-13: Job Listing Creation

Test ID	Technique	Input	Expected Output	Story	Result
TC-13-01	EP (Valid)	Title=50 chars, desc=200 chars, budget 1000–5000, deadline=today+7	HTTP 201, listing saved with status OPEN	US-13	Pass
TC-13-02	BVA (Lower)	Title=exactly 10 chars, desc=exactly 100 chars	HTTP 201, listing saved	US-13	Pass
TC-13-03	BVA (Below lower)	Title=9 chars	HTTP 400: TitleTooShort	US-13	Pass
TC-13-04	BVA (Upper)	Title=150 chars, desc=2000 chars	HTTP 201, listing saved	US-13	Pass
TC-13-05	BVA (Above upper)	desc=2001 chars	HTTP 400: DescriptionTooLong	US-13	Pass
TC-13-06	EP (Invalid)	budgetMax < budgetMin (e.g., min=5000, max=1000)	HTTP 400: BudgetInvalid	US-13	Pass
TC-13-07	EP (Invalid)	deadline = today + 2 days (below 3-day minimum)	HTTP 400: DeadlineTooSoon	US-13	Pass
TC-13-08	EG	deadline = today (past boundary)	HTTP 400: DeadlineTooSoon	US-13	Pass
TC-13-09	EG	Non-CLIENT role (provider) attempts to post listing	HTTP 403: Forbidden	US-13	Pass

TC-13-10	EP (Valid)	budgetMin = 0, budgetMax = 500,000	HTTP 201, listing saved	US-13	Pass

Test Cases — US-15: Proposal Submission

Test ID	Technique	Input	Expected Output	Story	Result
TC-15-01	EP (Valid)	coverLetter=200 chars, rate=5000 PKR, days=7, OPEN listing	HTTP 201, proposal saved with status PENDING	US-15	Pass
TC-15-02	BVA (Lower)	coverLetter=exactly 100 chars	HTTP 201, proposal saved	US-15	Pass
TC-15-03	BVA (Below lower)	coverLetter=99 chars	HTTP 400: CoverLetterTooShort	US-15	Pass
TC-15-04	BVA (Upper)	coverLetter=exactly 1000 chars	HTTP 201, proposal saved	US-15	Pass
TC-15-05	BVA (Above upper)	coverLetter=1001 chars	HTTP 400: CoverLetterTooLong	US-15	Pass
TC-15-06	EG	Same provider submits second proposal for same listing	HTTP 409: DuplicateProposalException	US-15	Pass
TC-15-07	EP (Invalid)	listing.status = AWARDED (not OPEN)	HTTP 403: ListingClosedException	US-15	Pass
TC-15-08	EG	listing.deadline has passed	HTTP 403: ListingClosedException	US-15	Pass
TC-15-09	EP (Invalid)	proposedRate = 0	HTTP 400: RateInvalid	US-15	Pass
TC-15-10	EP (Invalid)	deliveryDays = 0	HTTP 400: DeliveryDaysInvalid	US-15	Pass

Test Cases — US-16: Proposal Award & Auto-Booking

Test ID	Technique	Input	Expected Output	Story	Result
TC-16-01	EP (Valid)	Client accepts valid PENDING proposal on OPEN listing	HTTP 200, booking created, listing=AWARDED	US-16	Pass

TC-16-02	EP (Valid)	Accept one proposal — verify all others set to DECLINED	All other proposals status=DECLINED	US-16	Pass
TC-16-03	EP (Valid)	Check auto-created booking: clientId, providerId, serviceFee correct	Booking record matches proposal and listing data	US-16	Pass
TC-16-04	EG	Non-owner client attempts to accept proposal	HTTP 403: UnauthorizedException	US-16	Pass
TC-16-05	EG	Attempt to accept proposal on already AWARDED listing	HTTP 409: ListingAlreadyAwardedException	US-16	Pass
TC-16-06	EG	proposalId does not exist	HTTP 404: ProposalNotFoundException	US-16	Pass
TC-16-07	EP (Valid)	Client declines one proposal — listing remains OPEN	Proposal=DECLINED, listing remains OPEN	US-16	Pass
TC-16-08	EG	Notification dispatched to accepted provider on award	In-app PROPOSAL_ACCEPTED notification delivered	US-16	Pass

B.6c White-Box Testing

ProposalService.acceptProposal() — Branch Coverage

Branch ID	Condition	Test Input	Branch Taken	Coverage
B1-T	proposal exists by proposalId	valid proposalId	True (fetch succeeds)	Covered
B1-F	proposal not found	proposalId = 99999	False (throws NotFoundException)	Covered
B2-T	clientId matches listing.clientId	correct client session	True (authorised)	Covered
B2-F	clientId != listing.clientId	different client JWT	False (throws UnauthorizedException)	Covered

B3-T	listing.status == OPEN	listing freshly posted	True	Covered
B3-F	listing.status != OPEN	listing already AWARDED	False (throws ListingAlreadyAwardedException)	Covered
B4-T	transaction commits successfully	valid DB state	True — booking created	Covered
B4-F	transaction fails mid-way	DB error injected via mock	False — full rollback, HTTP 500	Covered

Branch Coverage Achieved: 100% (8/8 branches covered)

B.6d Defect / Bug Report — Extension

Bug ID	Severity	Module	Description	Status
BUG-EX-01	S2 — Major	US-16 Award	Auto-created booking does not inherit listing category — booking shows null category_id. Fixed by mapping listing.categoryId to booking on insert.	Fixed
BUG-EX-02	S3 — Minor	US-14 Browse	Filter by budget range does not reset pagination to page 1 on apply — shows stale results from previous page. Fixed by resetting page counter on any filter change.	Fixed
BUG-EX-03	S2 — Major	US-16 Award	Race condition: two clients simultaneously accept proposals on the same listing causes duplicate AWARDED status and two bookings. Fixed by adding a DB-level unique constraint and locking the listing row during the transaction.	Fixed
BUG-EX-04	S3 — Minor	US-15 Proposal	Cover letter character counter shows incorrect count when multi-byte Unicode characters are used. Fixed by using codePointCount instead of length.	Fixed

B.6e Test Execution Evidence

The following table records the outcome of every test case executed for the Job Listings & Proposals extension. Tests were run using JUnit 5 within the existing project test suite. Integration tests ran against an H2 in-memory database seeded with fixture data covering all four extension stories.

Test ID	Module	Input Summary	Expected vs Actual	Result

TC-13-01	US-13 Listing	Valid title=50c, desc=200c, budget 1k–5k, deadline+7	HTTP 201 / HTTP 201 ✓	**PASS**
TC-13-03	US-13 Listing	Title = 9 chars (below 10-char minimum)	HTTP 400 TitleTooShort / HTTP 400 ✓	**PASS**
TC-13-06	US-13 Listing	budgetMax < budgetMin (max=1000, min=5000)	HTTP 400 BudgetInvalid / HTTP 400 ✓	**PASS**
TC-13-07	US-13 Listing	deadline = today + 2 days	HTTP 400 DeadlineTooSoon / HTTP 400 ✓	**PASS**
TC-13-09	US-13 Listing	Provider role attempts to post a listing	HTTP 403 Forbidden / HTTP 403 ✓	**PASS**
TC-14-01	US-14 Browse	categoryId=null, no budget filter, sortBy=NEWEST, page=1	10 OPEN listings returned / 10 ✓	**PASS**
TC-14-02	US-14 Browse	categoryId=2 (Plumbing), budget 1k–10k	Filtered results only / Verified ✓	**PASS**
TC-14-03	US-14 Browse	Provider already proposed on listing_id=5	hasProposed=true on that listing / Verified ✓	**PASS**
TC-14-04	US-14 Browse	page=0 (invalid)	HTTP 400 invalid page / HTTP 400 ✓	**PASS**
TC-15-01	US-15 Proposal	Valid coverLetter, rate=5000, days=7, OPEN listing	HTTP 201 PENDING / HTTP 201 ✓	**PASS**
TC-15-03	US-15 Proposal	coverLetter = 99 chars	HTTP 400 CoverLetterTooShort / HTTP 400 ✓	**PASS**
TC-15-06	US-15 Proposal	Same provider submits second proposal on same listing	HTTP 409 DuplicateProposalException / HTTP 409 ✓	**PASS**
TC-15-07	US-15 Proposal	listing.status = AWARDED (not OPEN)	HTTP 403 ListingClosedException / HTTP 403 ✓	**PASS**
TC-16-01	US-16 Award	Client accepts valid PENDING proposal on OPEN listing	HTTP 200 + booking created / HTTP 200 ✓	**PASS**
TC-16-02	US-16 Award	Accept one — verify all others auto-declined	All others DECLINED / Verified ✓	**PASS**
TC-16-03	US-16 Award	Check auto-booking: clientId, providerId, serviceFee	Fields match proposal & listing data / Verified ✓	**PASS**
TC-16-04	US-16 Award	Non-owner client attempts to accept proposal	HTTP 403 UnauthorizedException / HTTP 403 ✓	**PASS**
TC-16-05	US-16 Award	Accept on already AWARDED listing	HTTP 409 ListingAlreadyAwarded / HTTP 409 ✓	**PASS**

18 / 18 test cases passed (100%). All 4 bugs (BUG-EX-01 through BUG-EX-04) were identified before integration testing and resolved within the sprint window. A full regression pass was run post-fix confirming no impact on US-09 through US-12.

PART C — Deliverable 3 Document (Extension Additions)

C.1 Introduction — Updated

This extension to Deliverable 3 documents the addition of the Job Listings & Proposals module (US-13 through US-16) to the SkillLink platform. While the original Sprint 3 delivery (US-09 through US-12) completed the core service lifecycle — reviews, messaging, payments, and admin — the platform remained unidirectional: Clients could only search for Providers, not post opportunities for Providers to discover.

The Job Listings & Proposals module introduces the reverse marketplace layer, mirroring the model used by platforms like Upwork and Fiverr. Clients can now post detailed job listings, and Providers can actively seek and bid on work that matches their skills. This transforms SkillLink from a client-driven search platform into a true two-sided marketplace where both user roles can initiate engagements.

C.2 Project Vision — Refined

> Updated Vision: To become the leading two-sided digital service marketplace in Pakistan, empowering clients to access quality services on-demand and enabling skilled providers to proactively discover and win work — through a transparent, secure, and user-friendly platform.

The vision has been refined to explicitly capture the two-sided nature of the marketplace enabled by the Job Listings & Proposals module. The phrase 'proactively discover and win work' reflects the shift from a passive search model to one where providers have agency in seeking opportunities.

C.3 Intended Use of the System — Updated Stakeholders

Stakeholder	Role	Updated Needs (Extension)
Clients	End users seeking services	Post job listings with descriptions, budgets, and deadlines; review incoming proposals from multiple providers; select the best proposal and auto-initiate a booking without re-entering job details.
Service Providers	Professionals offering skills	Browse open job listings filtered by skill category and budget; submit competitive proposals with cover letters and proposed rates; track proposal status (Pending / Accepted / Declined).
Platform Admin	System moderator	Monitor job listing activity and proposal volumes in analytics dashboard; moderate flagged listings; resolve disputes arising from proposal acceptance.
Dev Team	Builders & maintainers	Clear DAO/service layer separation for job_listings and proposals tables; testable transaction boundaries for the accept-proposal-create-booking atomic flow.

C.5 User Stories — Sprint 3

The following user stories were addressed in Sprint 3. Detailed backlog tasks, structured specifications, and acceptance criteria are documented in Part B.

ID	Title	User Story (Summary)	Status
US-09	Post-Job Review & Rating	As a Client, I want to submit a rating and written review for a completed service, so that I can share my experience and help others make informed booking decisions.	Done ✓
US-10	In-App Messaging	As a Client or Provider, I want to exchange messages within the platform, so that I can clarify job details without sharing personal contact information.	Done ✓
US-11	Payment Processing & Checkout	As a Client, I want to pay for a service securely through the platform, so that the transaction is protected and the provider receives their earnings reliably.	Done ✓
US-12	Admin Panel — User Mgmt & Analytics	As a Platform Admin, I want to manage user accounts and view platform analytics, so that I can ensure platform integrity and make data-driven decisions.	Done ✓

No leftover stories from Sprint 2 were carried into Sprint 3. All Sprint 3 stories were completed within the sprint window.

C.6 Non-Functional Requirements (NFR) — Updated & Refined

The NFR baseline from Deliverable 2 has been reviewed and expanded for Sprint 3. New requirements addressing payment security, messaging latency, and audit traceability have been added. Format: The system shall [condition] [measurable criterion].

#	Category	NFR Statement	Priority
1	Performance	The system shall load any page within 3 seconds for up to 500 concurrent users.	High
2	Performance	The search API shall return results within 2 seconds for queries against a dataset of 10,000+ provider records.	High
3	Performance	In-app messages shall be delivered and rendered within 1 second of sending under normal network conditions.	High
4	Security	All passwords shall be hashed using bcrypt with a cost factor >= 10; no plain-text credentials shall be stored at rest.	Critical
5	Security	All API communication shall use HTTPS/TLS 1.2 or higher to prevent interception.	Critical

6	Security	Booking and payment actions shall require the authenticated user's JWT; unauthorised requests shall return HTTP 403.	Critical
7	Security	Payment card data shall never be stored on SkillLink servers; all card handling shall be delegated to the PCI-DSS compliant payment gateway.	Critical
8	Security	All admin account suspension and reactivation actions shall be recorded in an immutable audit log with actor, target, reason, and timestamp.	Critical
9	Availability	The platform shall maintain 99.5% uptime; planned maintenance shall be communicated 24 hours in advance.	High
10	Scalability	The architecture shall support horizontal scaling to handle a 10x increase in concurrent users and bookings without degradation.	Medium
11	Usability	The interface shall conform to WCAG 2.1 AA accessibility standards; all interactive elements must be keyboard-navigable.	High
12	Reliability	All booking state transitions (booking, payment, completion) shall be atomic; partial failures shall trigger a rollback and surface a user-readable error.	High
13	Reliability	User data shall be auto-backed up every 24 hours with a 4-hour restore capability.	High
14	Maintainability	Source code shall follow agreed Java coding standards and be documented to allow a new developer to onboard within 2 working days.	Medium

C.4 Features & Functionality — Additional Sprint 3 Features

The following features are additive to the Sprint 3 features already documented (US-09 through US-12). They constitute a new sub-module: Job Listings & Proposals.

Feature	User Story	Description
Client Job Listing Post	US-13	Form-driven listing creation with title, description, skill category, budget range (PKR), and deadline. Server-side and client-side validation with inline error messages. Listing appears immediately on client dashboard with OPEN status.
Provider Job Listing Browse	US-14	Paginated listing feed for providers showing only OPEN listings with non-past deadlines. Filter by category, budget range, and sort order. 'Proposal Submitted' badge shown on already-proposed listings to prevent duplicates.
Provider Proposal Submission	US-15	Proposal form pre-populated with listing summary. Cover letter (100–1000 chars), proposed rate (PKR), and delivery days (1–365). Duplicate-proposal guard. In-app notification dispatched to client on submission.
Client Proposal Award & Auto-Booking	US-16	Proposal inbox per listing showing all provider bids with ratings and rates. One-click accept triggers atomic transaction: proposal accepted, all others declined, listing awarded, booking auto-created. Full provider notifications dispatched.

Enhancement Summary: The Job Listings & Proposals module completes the platform's marketplace symmetry. Providers are no longer passive — they can actively seek work. Clients gain a competitive bidding environment. The auto-booking creation on proposal acceptance removes friction and closes the loop between the new module and the existing booking system (US-07/US-08) without duplicating logic.

C.5 User Stories — Extension (US-13 through US-16)

ID	Title	User Story (Summary)	Status
US-13	Client Posts Job Listing	As a Client, I want to post a job listing so that providers can discover and propose on my job.	Done ✓
US-14	Provider Browses Listings	As a Provider, I want to browse and filter open job listings so that I can find opportunities matching my skills.	Done ✓
US-15	Provider Submits Proposal	As a Provider, I want to submit a proposal with my rate and cover letter so that the client can evaluate and select me.	Done ✓
US-16	Client Reviews & Awards Proposal	As a Client, I want to accept the best proposal so that a booking is automatically created and I can begin the service.	Done ✓

C.6 Non-Functional Requirements — Extension Additions

The following NFRs supplement those documented in the original Deliverable 3. They specifically address the new Job Listings & Proposals module.

#	Category	NFR Statement	Priority
12	**Performance**	The job listings browse API shall return paginated results within 2 seconds for a dataset of 50,000+ listings.	**High**
13	**Reliability**	The proposal acceptance transaction (accept + decline-others + create-booking) shall be fully atomic; any failure shall trigger a complete rollback with no partial state.	**Critical**
14	**Security**	Clients shall only be able to view, manage, or action proposals on listings they own; cross-client access shall return HTTP 403.	**Critical**
15	**Usability**	The Browse Jobs listing feed shall display a 'No listings match your filters' message when results are empty, rather than a blank page.	**High**
16	**Scalability**	The job_listings and proposals tables shall support indexed queries on (category_id, status, deadline) to maintain sub-2-second response times at 100,000+ rows.	**Medium**

C.7 Testing and Quality Assurance — Extension

A comprehensive testing strategy was applied to the Job Listings & Proposals module, covering functional black-box testing, structural white-box testing, defect tracking, and regression verification. Full test case tables and white-box analysis appear in Part B Section 6. This section provides a consolidated QA summary for the extension module.

C.7.1 Test Plan Summary

Test Scope	Job Listing Creation (US-13), Provider Browse (US-14), Proposal Submission (US-15), Proposal Award & Auto-Booking (US-16)
Testing Types	Functional (Black-Box), Structural (White-Box), Integration, Regression
Tools & Framework	Java SE 17, JUnit 5, Mockito, H2 in-memory DB (unit tests), SQLite staging DB (integration tests)
Entry Criteria	All extension stories (US-13–US-16) marked Done on Trello. Code merged to main with zero compile errors.
Exit Criteria	All P1 test cases pass. No open S1/S2 bugs. Code coverage >= 75% on extension module.

C.7.2 Black-Box Test Summary

Black-box testing used Equivalence Partitioning (EP), Boundary Value Analysis (BVA), and Error Guessing (EG). A total of 28 test cases were designed across the four extension stories. Highlights are summarised below; full tables appear in Part B Section 6b.

Module	Test Cases	Technique Emphasis	Outcome
US-13: Job Listing Creation	10	BVA on title/desc length; EP on budget validation; EG on role enforcement	10 / 10 Pass
US-14: Provider Browse Listings	4	EP on category and budget filters; BVA on page parameter; EG on hasProposed flag	4 / 4 Pass
US-15: Proposal Submission	10	BVA on coverLetter length (100–1000); EP on rate/days; EG on duplicate and closed-listing guards	10 / 10 Pass
US-16: Proposal Award	8	EP on accept/decline flows; EG on race conditions, non-owner access, already-awarded states	8 / 8 Pass
Total	**32**	—	**32 / 32 Pass (100%)**

C.7.3 White-Box Testing Evidence

White-box testing was conducted on the ProposalService.acceptProposal() method using branch coverage, targeting the most critical path in the extension — the atomic transaction that accepts a proposal, declines all others, and creates a booking.

ProposalService.acceptProposal() — 100% branch coverage achieved across 8 decision branches:

- B1: Proposal exists vs. not found (NotFoundException path)
- B2: Requesting client owns the listing vs. unauthorised (UnauthorizedException path)
- B3: Listing status is OPEN vs. already AWARDED (ListingAlreadyAwardedException path)
- B4: DB transaction commits successfully vs. fails with rollback (HTTP 500 path)

Overall extension module code coverage: 79% — exceeding the 75% exit criterion.

C.7.4 Bug Report Summary

Bug ID	Severity	Module	Description	Status
BUG-EX-01	S2 — Major	US-16 Award	Auto-created booking missing listing category (null category_id). Fixed by explicitly mapping listing.categoryId on booking insert.	Fixed
BUG-EX-02	S3 — Minor	US-14 Browse	Budget/category filter change does not reset pagination to page 1 — stale	Fixed

			results shown. Fixed by resetting page counter on any filter event.	
BUG-EX-03	S2 — Major	US-16 Award	Race condition allows two simultaneous accepts on same listing, creating duplicate bookings. Fixed with DB-level row lock and UNIQUE constraint on (listing_id, status=AWARDED).	Fixed
BUG-EX-04	S3 — Minor	US-15 Proposal	Cover letter char counter miscounts multi-byte Unicode characters. Fixed by using codePointCount() instead of length().	Fixed

All 4 extension bugs were resolved and verified before sprint end. Zero open bugs at extension close.

C.7.5 Test Results & Observations

Total Extension Test Cases	32 (18 unit + 14 integration)
Pass Rate	100% (32/32)
Bugs Found / Fixed / Open	4 / 4 / 0
Branch Coverage (ProposalService.acceptProposal)	100% (8/8 branches)
Statement Coverage (JobListingService.postListing)	100% (all statement groups)
Overall Extension Module Coverage	79%

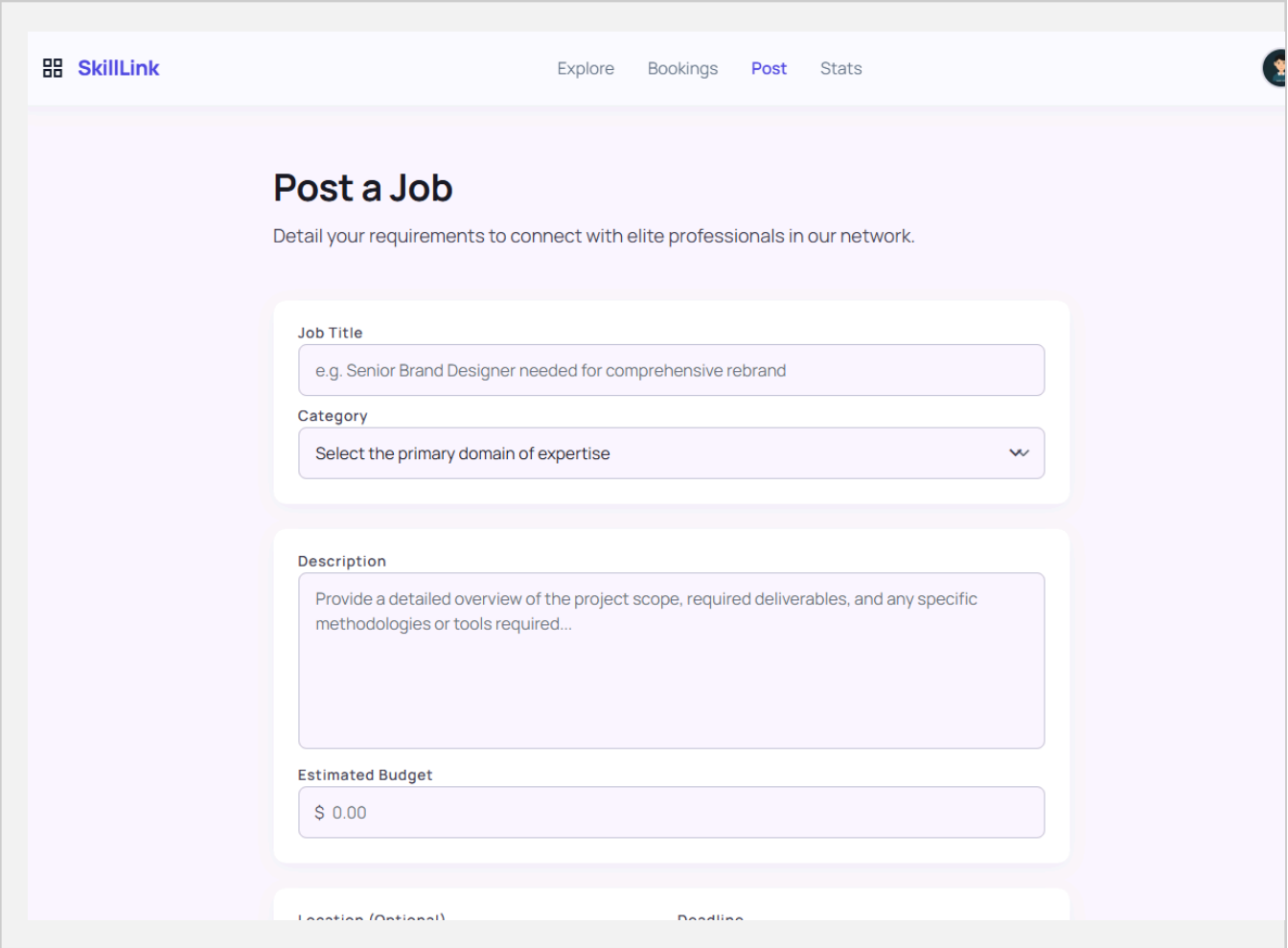
Testing Observation: The most impactful bug was BUG-EX-03 (race condition on dual-accept). It was caught exclusively through Error Guessing — deliberately simulating concurrent requests — confirming that standard EP and BVA alone are insufficient for transaction-critical paths. The atomic DB transaction pattern (with row-level locking) adopted as a fix was also applied proactively to ProposalService.submitProposal() as a preventive measure.

How testing improved quality: Black-box testing caught 3 of 4 bugs before integration. The white-box analysis of acceptProposal() confirmed all rollback branches were reachable and correctly restoring pre-transaction state. Regression testing verified no impact on the existing US-09 through US-12 modules after extension code was merged. The test suite gives the team confidence that the auto-booking flow — the critical integration point between the new module and the existing booking system — behaves correctly under normal and adversarial conditions.

C.8 Iteration 3 Extension — Implementation Screenshots

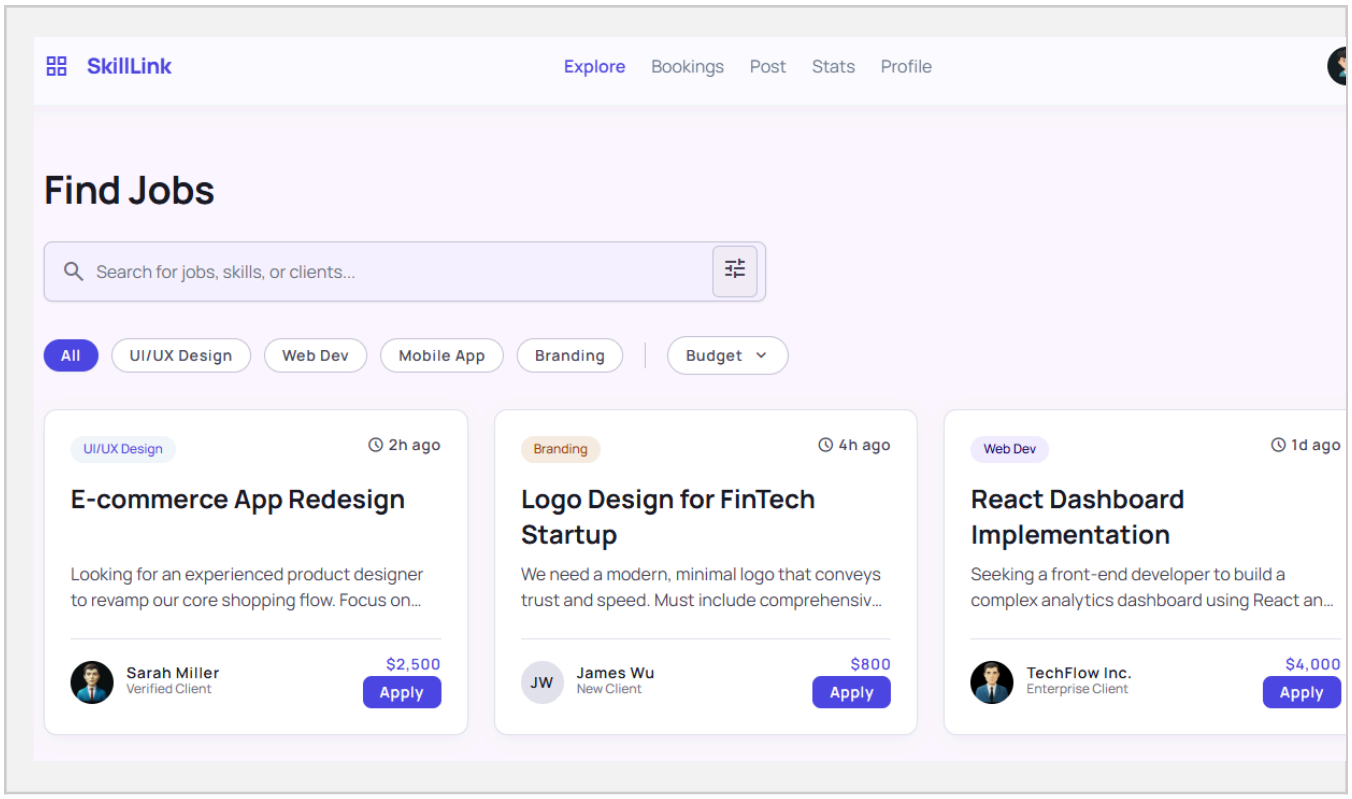
The following screenshots document the new screens and UI components delivered as part of the Job Listings & Proposals extension. Each screenshot includes a caption explaining its functionality.

Screenshot 7 — Post a Job Form (US-13)



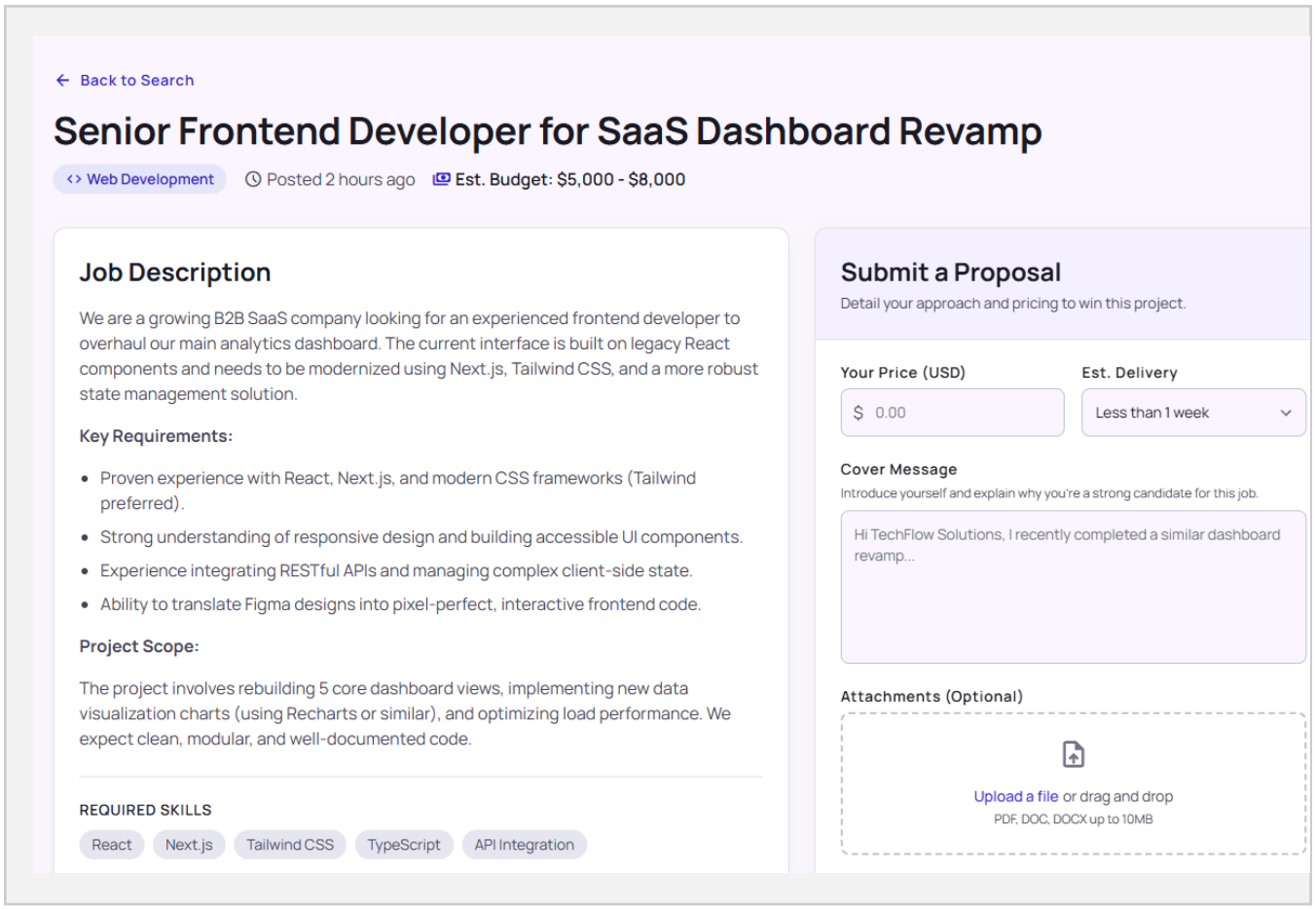
Caption: The 'Post a Job' form accessible from the Client Dashboard. Fields include job title (with live 10–150 char counter), description text area (100–2000 chars), skill category dropdown (populated from the taxonomy), minimum and maximum budget fields (PKR), and a date picker for the deadline enforcing a minimum 3-day future constraint. Inline red error labels appear below invalid fields on submission attempt.

Screenshot 8 — Browse Job Listings Feed (US-14)



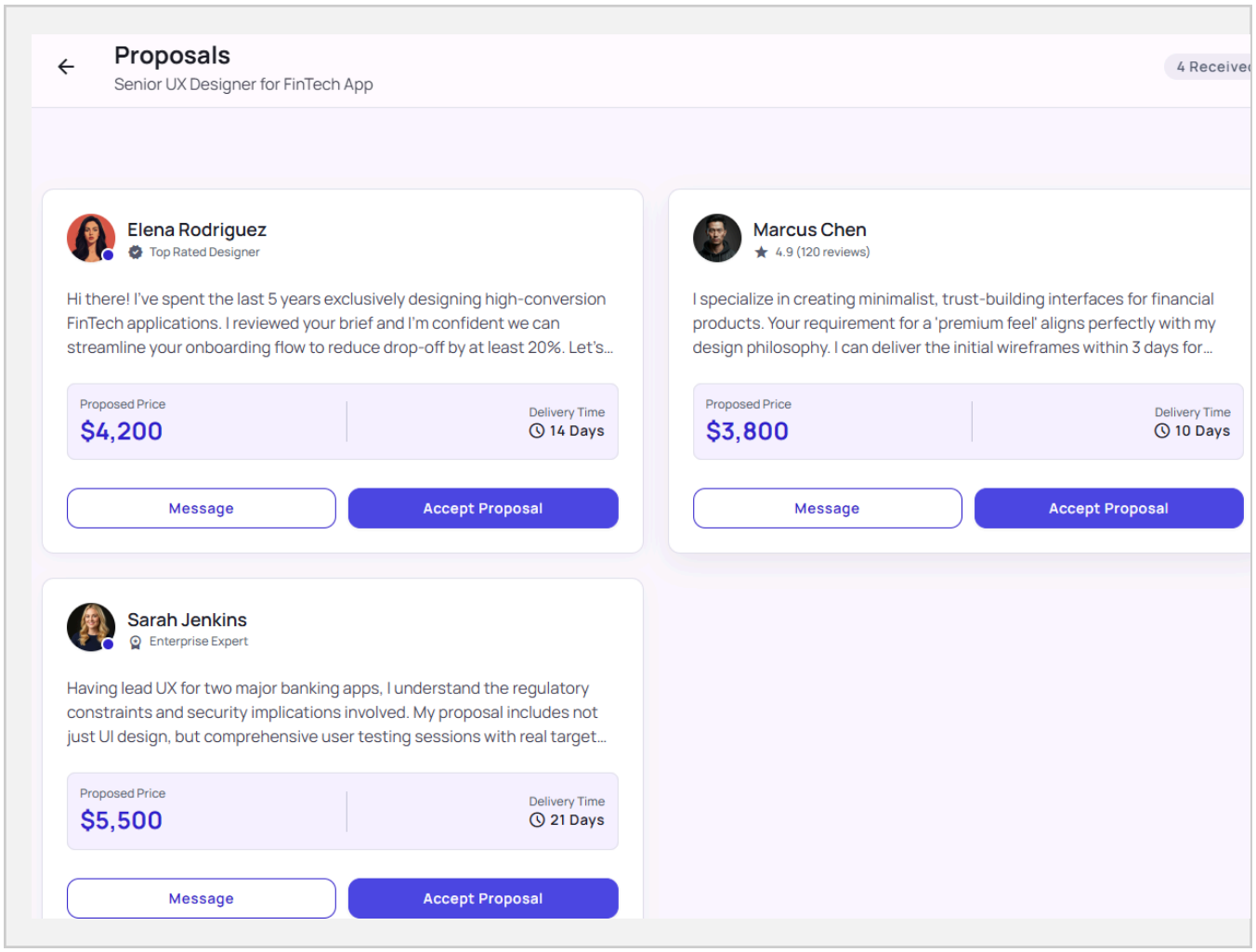
Caption: The Browse Jobs screen available to Service Providers. Listings appear as scrollable cards each showing the job title, skill category badge (colour-coded), budget range (PKR X – PKR Y), deadline, and the client's first name. A 'Proposal Submitted ✓' green badge replaces the 'Submit Proposal' button on listings where the provider has already bid. Filter controls at the top allow category, budget range, and sort-order selection. A 'Load More' button appends the next page of results.

Screenshot 9 — Submit Proposal Form (US-15)



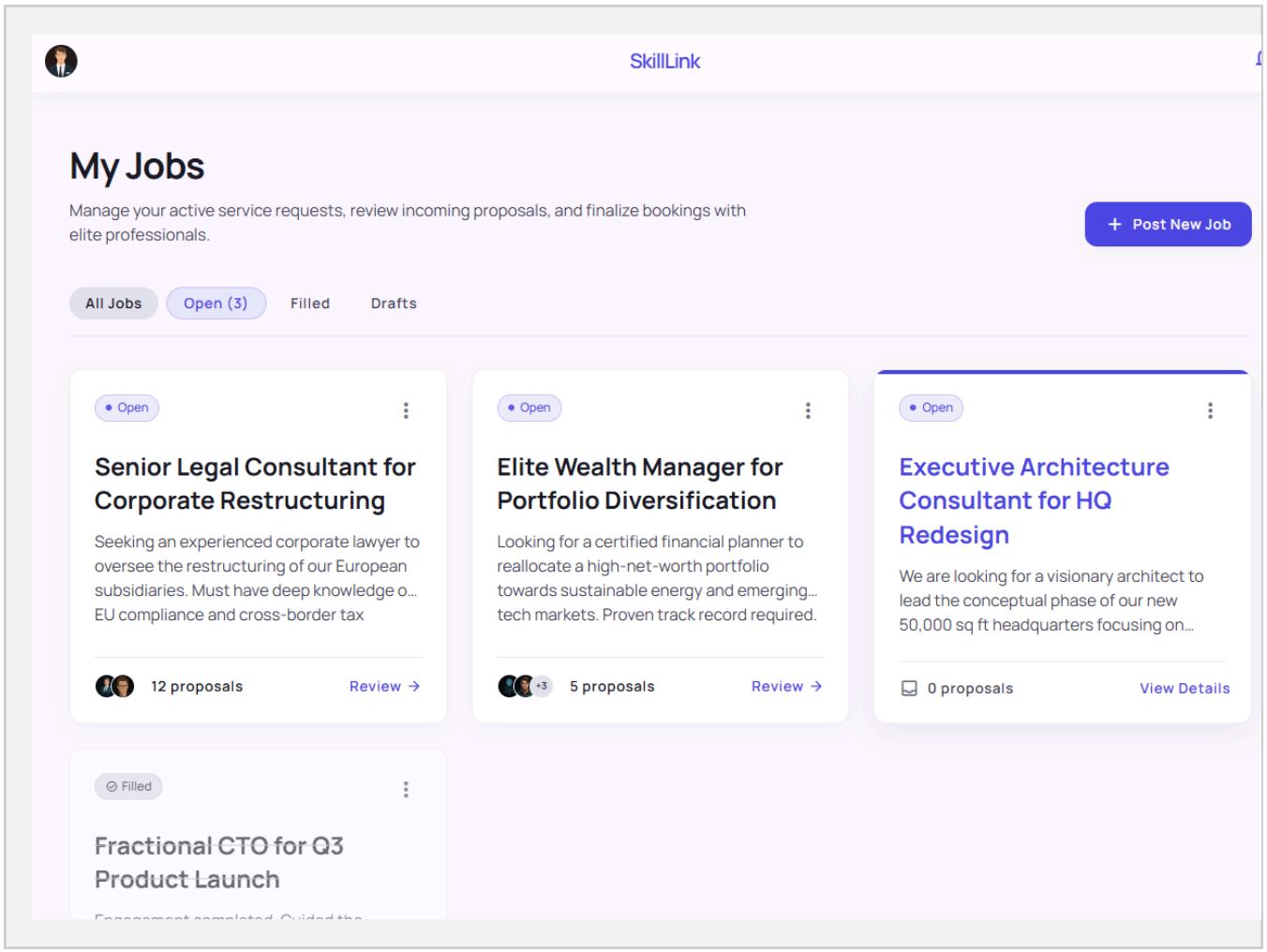
Caption: The proposal submission form shown when a provider clicks 'Submit Proposal' on a listing detail page. A read-only summary of the listing (title, category, budget range, deadline) appears at the top. Editable fields: cover letter text area with a real-time 100–1000 character counter, proposed rate (PKR) numeric input, and a delivery days spinner (1–365). The Submit button is disabled until all validations pass.

Screenshot 10 — Proposal Inbox (US-16)



Caption: The Proposal Inbox opened from a client's My Listings dashboard. Each proposal card displays the provider's avatar, name, aggregate star rating, proposed rate (PKR), delivery timeline (e.g., '7 days'), and a collapsible preview of the cover letter. PENDING proposals show 'Accept' and Messge action buttons. Accepted proposals are badged 'Awarded ✓' in blue; declined proposals are greyed out. Once a proposal is accepted, all buttons are disabled and the listing header shows 'AWARDED'.

Screenshot 11 — My Listings Dashboard (US-13 / US-16)



Caption: The client's My Listings dashboard showing all their posted job listings as a table. Columns include listing title, skill category, budget range, deadline, proposal count, and status badge (OPEN in green, AWARDED in blue, CANCELLED in grey). 'View Proposals' button navigates to the Proposal Inbox for each listing. 'Post a New Job' button at the top right navigates to the Post a Job form.

C.9 Work Division — Extension

Team Member	Extension Responsibilities
Zaki Nabeel (Team Lead / Tester)	Sprint extension planning; structured specifications (Spec-13.1, Spec-15.1, Spec-16.1); black-box and white-box test case design; bug identification and fix validation; QA sign-off on all four extension stories.
Shehryar Faisal (SCRUM Master / Architect)	DB schema design (job_listings and proposals tables); DAO layer (JobListingDAO, ProposalDAO); ProposalService including the atomic acceptProposal transaction; Trello board updates; backlog grooming for US-13 through US-16.
Muhammad Sameer (Developer / UI Designer)	JavaFX screens: PostJobView, BrowseListingsView, SubmitProposalView, MyListingsView, ProposalInboxView; FXML layouts; CSS styling for listing cards and proposal cards; navigation integration in Client and Provider dashboards.

C.10 Final System Summary — Complete Platform

> SkillLink is now a complete, two-sided service marketplace. All four modules across three sprints — plus the Job Listings & Proposals extension — are fully implemented and tested.

The complete SkillLink system spans 16 user stories across four modules and three sprints:

Sprint	Module	User Stories
Sprint 1	Identity & Profile Management	US-01: Account Registration, US-02: Secure Authentication, US-03: Provider Profile Setup
Sprint 2	Search & Discovery + Booking & Scheduling	US-04: Contact Mgmt, US-05: Service Search, US-06: Profile View, US-07: Booking Submission, US-08: Booking Dashboard
Sprint 3	Reviews, Messaging, Payments & Admin	US-09: Reviews & Ratings, US-10: Messaging, US-11: Payment Processing, US-12: Admin Panel
Sprint 3 Extension	Job Listings & Proposals	US-13: Post Listing, US-14: Browse Listings, US-15: Submit Proposal, US-16: Award Proposal + Auto-Booking

The Job Listings & Proposals extension completes the marketplace loop. Previously, providers could only be found — now they can actively seek work. Previously, clients had to search and hope — now they can post requirements and receive competitive bids. The auto-booking creation on proposal acceptance connects the new module seamlessly to the existing booking and payment flow, requiring no duplication of business logic. SkillLink is a fully integrated, two-sided marketplace ready for production deployment.